Technical report — Polyglot Explainer

> ***Status: Day-3 draft (2026-10-12).*** *Content below this header describes real, verifiable work. Evaluation results for the live endpoint (§5), the demo video URL and final model/endpoint choices are* ***not yet decided/measured*** *and are marked* `[TBD]` *— nothing here is fabricated.*

- **Track:** Track 2B — Own Project
- **Event:** Hack Apertus Online Hackathon, Oct 12–15 2026
- **Team:** `arin-dewangan` (solo entry)
- **Deliverable PDF filename (mandated):** `arin-dewangan_Report.pdf` (rendered from this document Day 3, 2026-10-12, as `docs/arin-dewangan_Report.pdf`; ≤6 pages of body content)
- **Demo:** `docs/demo-video.mp4` (rendered Day 2, 2026-10-12; ffprobe-verified Day 3: 110.0 s, 1280×720, h264+AAC ≤2:00; public URL TBD at submission)
- **Repo:** `[TBD — created from the HackApertus project-template on Oct 12, public]`

1. Summary

Polyglot Explainer is a multilingual explainer web app powered by Apertus, Switzerland's sovereign open LLM. The user types any complex topic, picks a language (13 seed languages: en, hi, ta, bn, kn, te, ml, mr, gu, fr, de, ja, sw — the model itself covers 1,811), and picks a depth (ELI5 / standard / deep); the app generates an explanation **directly in the requested language** via an Apertus model behind an OpenAI-compatible endpoint. The project's value proposition is sovereign, vendor-neutral multilingual explainability: a single Docker image (`make run`) that serves a working product with no required keys, while a real key unlocks live Apertus inference through any OpenAI-compatible endpoint (Infomaniak, Swisscom, or a local Ollama/vLLM instance). Stack: Flask + `requests` client + gunicorn, served from `track_2b/src/app.py`; requests are cached in-memory on `(topic, language, depth)` so repeated queries cost nothing.

2. Architecture

Request flow: browser UI (`GET /`) → Flask app → validation → in-memory cache lookup → either the OpenAI-compatible chat-completions client (`POST {LLM_BASE_URL}/chat/completions` with the configured `LLM_NAME`) or the env-gated **demo mode** (canned, clearly-labeled `[SIMULATED DEMO OUTPUT]` strings). `POST /api/explain` is the documented input/output contract (the Track 2B "specified format"):

- Request: `{"topic": string (required, non-empty), "language": string (ISO 639-1, one of the 13 seed codes), "depth": "eli5" | "standard" | "deep"}`
- Response (200): `{"topic", "language", "depth", "model", "demo", "cached", "explanation"}`
- Errors: `400` on invalid input (message names the bad field); `502` when the upstream LLM call fails. Also `GET /api/languages` (200, 13 languages) and `GET /healthz` (200, `{"status","demo","model"}`).

Docker: `track_2b/Dockerfile` (python:3.12-slim, `pip install` from `src/requirements.txt`, gunicorn binding `0.0.0.0:5000`) + `docker-compose.yml` (ports `${PORT:-5000}:5000`, `env_file: .env` optional, `LLM_*` wired through `environment:`) + `track_2b/Makefile` `run: docker compose up --build` (+ repo-root delegate for `make run` at repo root). Demo mode means `make run` succeeds end-to-end on a clean checkout with no secrets and no network beyond

the image build — the judges' default path.

Target architecture (mandatory)

Declared: (a) on-premise as primary — a single Docker image served on the organisation's own infrastructure; the LLM is reached through any OpenAI-compatible endpoint, including a self-hosted Ollama/vLLM instance, so there is no vendor lock. When the endpoint is Infomaniak's AI product, the same deployment additionally maps to **(c) sovereign Swiss cloud** (Swiss-hosted, Swiss jurisdiction). We do **not** claim (b) air-gapped: the live path needs the model endpoint, and the no-network path is honest demo-grade canned output, not bundled weights.

3. Use of Apertus

• **Model:** `swiss-ai/Apertus-v1.5-70B` (default `LLM_NAME`; the 8B variant is the documented fallback). **[TBD — final live choice depends on which inference key materialises: Infomaniak free tokens → Discord/Public AI → local Ollama GGUF-8B.]**

• **How it is used:** inference only (chat completions). No fine-tuning, no training. **Where it runs:** remote hosted endpoint or local Ollama — the choice is purely an env var (`LLM_BASE_URL`).

• **Prompt strategy (real):** system prompt instructs the model to write the ENTIRE explanation in the requested language with no English preamble and no meta-commentary; the user prompt carries depth-specific guidance and length caps (ELI5 ≈100–150 words / 600 tokens; standard ≈250–400 / 1200; deep ≈500–800 / 2200). Multilinguality comes from the Apertus model generating natively in-language, not translate-after-generation.

• **Demo mode (honesty note):** when `LLM_API_KEY` is absent, the env-gated client serves canned outputs; every demo output is prefixed `[SIMULATED DEMO OUTPUT — no live model was called]` and the JSON carries `"demo": true`. Demo outputs are showcase-only and must never be cited as model quality (see §5).

• **Other models:** none used in the product. `[TBD — if an automatic judge is used during evaluation, it will be disclosed here per template rules.]`

4. Data

`data/` currently holds `sample_topics.json` (15 topics + the 13 seed languages + 3 depths — the demo/eval fixture list) and `eval_cases.json` (the planned evaluation matrix, ≤1 KB each; total `data/` < 10 KB, far below the 100 MB cap). No training data, no fine-tuning, no personal data. `[TBD — an optional public evaluation dataset (HackApertus HF dataset card: eval set + model responses + metadata) will be decided on Oct 13–14.]`

5. Evaluation

Status (Day 2): baseline measured (demo mode); live-endpoint spot-check planned once an inference key exists. Planned live check: a small self-made spot-check (5 topics × 6 languages = 30 queries) run against the LIVE endpoint, judged by the builder against three binary criteria: (1) explanation entirely in the requested language, (2) appropriate depth/length, (3) no factual howlers on spot-check. Metrics tied to judging criteria: language fidelity (purposeful use of AI), cache-hit latency + 400/502 error-path correctness (technical rigour). Results table will be filled after the live key is available.

Setup	Metric	Result
Baseline (demo mode)	all UI/API paths return labeled simulated output	PASS — **13/13 pytest green** (Day 1; re-validated 13/13 green Day 3), Flask smoke test all green (re-validated Day 3): `GET /` 200 + demo badge, `GET /api/languages` 200 (13 langs), `POST /api/explain` 200 with `demo:true`, repeat → `cached:true`, bad input → 400, `GET /healthz` 200
Live Apertus	language fidelity / depth / spot-check accuracy (30 cases)	`[TBD — pending inference key]`

Demo-mode outputs are canned and labeled; they are excluded from any model-quality claim.

Cost & latency (grounded in the implementation, not yet measured live)

- **Per-request upstream cost:** exactly one `POST {LLM_BASE_URL}/chat/completions` per cache miss (`_live_explain`); cache hits (`(topic, language, depth)` in-memory key, FIFO trim at 200) make **zero** upstream calls — repeated queries cost nothing and return instantly.
- **Token budget caps (code):** `max_tokens` 600 (eli5) / 1200 (standard) / 2200 (deep), temperature 0.7 — the depth dial directly bounds spend.
- **Demo-mode latency:** local-only (Flask response, no network) — sub-second by construction; measured informally during the smoke test, not benchmarked.
- **Live latency/cost:** endpoint-dependent (70B vs 8B tradeoff) and **not yet measured** — no inference key exists at Day 2. The client sets a 120 s request timeout (`APERTUS_TIMEOUT`).
- **Scaling:** stateless Flask + gunicorn; the only shared state is the per-container cache, so horizontal scaling is trivially safe (cache is a pure optimisation).

6. Limitations

Honest, verifiable limits: (1) Without `LLM_API_KEY`, all outputs are canned simulated demos — useful for UX judging, not model judging. (2) Prompt-level multilinguality is not a guarantee for low-resource languages; the live eval will say how far the 70B model really goes. (3) Cache is in-memory and single-container (lost on restart). (4) No auth, no multi-user, no streaming — one stateless Flask service. (5) Live latency/cost depend on the chosen endpoint (70B vs 8B tradeoff). (6) The `make run` container boot has not yet been executed on a Docker-capable host (this VM cannot run containers — nested restriction); the Dockerfile is deliberately conservative and `make run` end-to-end verification is scheduled on a capable machine.

7. Reproducibility

Judges' path (no key): `git clone <repo> && cd track_2b && make run` → open `http://localhost:5000/` (200, demo badge) and `POST /api/explain {"topic":"photosynthesis","language":"hi","depth":"eli5"}` (200, `"demo": true`). With a key: `cp .env.example .env`, fill `LLM_NAME`/`LLM_BASE_URL`/`LLM_API_KEY`, re-run. Local Ollama fallback: `LLM_BASE_URL=http://host.docker.internal:11434/v1`, `LLM_API_KEY=ollama`. Demo video: `docs/demo-video.mp4` rendered Day 2 (2026-10-12, real app footage in labeled demo mode + synthesized original music); ffprobe-verified Day 3: 110.0 s (≤2:00), 1280×720, h264+AAC — public URL `[TBD — upload on submission, by Oct 16 12:00 CEST]`. Repo URL: `[TBD]`.

8. Next steps

1. **Streaming responses** (SSE) for perceived latency — value/UX.
2. **Persistent cache** (Redis) so the cost story survives restarts — value, cost & scalability.
3. **Eval harness with a disclosed automatic judge** (e.g. LLM-as-judge for language fidelity) — technical rigour.
4. **Infomaniak** → **Ollama failover** in the client for offline resilience — sovereign deployability.
5. **More languages/depths + prompt A/B notes** — purposeful use of AI.
6. Optional public evaluation dataset release.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. (Code: Apache 2.0, see repo-root LICENSE.)

References

• Apertus model collection (Hugging Face, `swiss-ai` org): <https://huggingface.co/collections/swiss-ai/apertus-llm>
• Apertus paper (pre-print): <https://arxiv.org/abs/2509.14233>
• Apertus docs overview (swiss-ai/www-apertus): <https://github.com/swiss-ai/www-apertus/blob/HEAD/content/docs/overview.md>
• HackApertus project template: <https://github.com/HackApertus/project-template> (cloned 2026-10-12, HEAD `09f8e9e2b588db3325869b23c6d5132f5c3756f7`)